

Bridge Methods: Complementary Steps Integrating Agile Development Tools & Methods with Formal Process Methodologies

Stephen J. Cohen
Microsoft Corporation

William H. Money
The George Washington University

Abstract

This paper describes a “Bridge” method for successfully integrating Agile concepts with traditionally linear and sequential software development lifecycles. A software development and management environment is constructed to serve project oversight, management control, and team planning and task scheduling needs. The new method and environment enable project stakeholders to acquire a clear and constructive understanding of each other’s activities. The method is neither a tentative step toward Agile nor a ‘maturing’ of an already Agile environment. Instead the Bridge is a complementary destination methodology incorporating the techniques from both Agile and Formal methods that have proven effective in large, complex software delivery environments. The method and environment are assessed within a large enterprise integration and software development project, and refined using an Action research methodology. This work advances the thinking about Agile’s usefulness and its ability to effectively address multiple stakeholder needs for large enterprise development projects.

1. Introduction: Defining the problem space

Many organizations apply linear sequential software development models traditionally following a classic life cycle or waterfall model to control software development projects, develop task plans, assign responsibility, staff the work, and carefully manage budgets. The overall status of projects are then driven by a managerially understood and accepted milestone reporting process that supports performance reporting lauded by project managers, and required by organizationally approved and adopted management standards. The actual development work processes are similarly driven through progressive engineering phases broadly grouped into requirements analysis, modeling, design, coding, testing, and implementation/supporting categories. [12]

Management has adopted this phased method and its associated processes and control framework for many reasons. One obvious factor promoting the adoption is

the existence of public and historically available well known standards and guidelines based on ISO 9001 such as ISO/IEC 12207 and ISO/IEC 15288. The readily available management frameworks match the standards and are widely disseminated. They are mandated for adoption on many government projects where MIL-STD-498 and MIL-STD-499B/ANSI/EIA-632 are predetermined as the system and software development implementation framework, and SEI CMMI to level 3 as the key processes. Finally, they are frequently supported by formalized organizational policies that dictate that tailored SEI CMM level 3 (or higher) processes be utilized on enterprise projects.

However, there are well researched difficulties with this single model of representing both development and management tasks. The literature indicates that users often modify or discover their requirements as organizations grow, evolve and meet specific user needs. [6][11] Changes may not overly stress developers who will readily modify their work “on-the-fly” and substitute new tools and work steps needed to complete a project while appropriately adjusting their time and schedules to achieve performance objectives. Such changes frequently require major modifications to many portions of the formal documentation, work plan changes, and skip or modify traditionally described development processes thereby creating havoc with the entrenched project description. da Silva and da Cunha [10] characterize this as a paradox of trying to reconcile Agile methods with the formality and structures of the management processes. They described their work in one organization to develop a process (methodology) to manage evolving requirements by tailoring the development practices based upon selected portions of eXtreme Programming, Adaptive Software Development, and Scrum. The approach they describe is tailored for use when unstable requirements of the customer are major factors in the development of the system. Their methodology has four phases. They are:

Requirements Phase; Dedicated to project preparation and specification of high-level requirements.

Design, Engineering and Validation Phase. Where development occurs iteratively.

Acceptance Phase. Where acceptance tests are performed by the customer before closure.

Operations and Maintenance Phase. Eventual bugs and fixes are addressed in this phase. [10]

Following da Silva and da Cunha's methodology, these phases are implemented using milestone meetings where key tasks and objectives are validated, and where it is determined if the project is ready to move to the next phase.

A number of other authors have addressed this known conflict and difficulty associated with implementing philosophically Agile processes inside of programs requiring these inherently traditional and process driven methodologies. [5][7][9] This paper describes a hybrid "Bridge" technique for software design and development that expands on the tools, processes, and steps required to bridge the gap. The Bridge approach is highly useful because it ensures that the organization will be able to obtain the predictability, transparency, and consistency that the clearly understood traditional processes promise to large complex programs. The over-sight provided by the traditional model is required to minimize project risk while creating an environment in which the process follows the workers, not the other way around. Agile and traditional methods are synergistic when used to create multiple supporting and hypothetically self-validating views of the process appropriate to the perspective of the observer. External governing entities and project stakeholders can rely on the large traditional process milestones, structured periodic reviews, and comparative metrics that have traditionally been applied to form opinions of project completeness, quality, and overall status. At the same time, program management can use a more detailed set of measures which combine the derived metrics from the Agile processes with the more Formal practice of managing trends against a known baseline such as Six Sigma. [14] Lastly, the project teams can use the immediate feedback loops aligned with Agile methods to reduce time to market and increase product quality.

The problem addressed by the presence of different models is one that involves clearly communicating and developing a shared understanding of the different purposes of each view of the project by all of the involved parties. The problem is summarized in an analogy published by Loren Eiseley. [8] Eiseley's example describes a lesson he says he learned from a spider. The spider lived in her web, a universe that she understood and one in which she felt and knew all that could happen "...the tug of wind, the fall of a raindrop, the flutter of a trapped moth's wing." Eiseley describes

how he used a pencil to move her web, and observed the inaction of the spider since the movement was not a familiar one that could be comprehended and well understood like the struggle of a captured prey. He indicates that the pencil point is "... an intrusion into this universe" without precedent. The spider understood the spider's universe, and the intrusion was simply not explainable. This model applies broadly to the difficulties of achieving a shared vision and strong communication between the groups of Agile programmers and engineers, project managers, and stakeholders. Neither sees the web of the other well, and the requirements of each are foreign in purpose without readily mapped relationships. The communication difficulty becomes particularly acute with management's legitimate authoritative demand that the Agile steps and methods somehow be forcefully mapped into traditional planning and reporting channels. The authors hypothesize that the bridge methodology can aid the different users greatly by creating an understanding of the different purposes of each perspective, and providing a strong shared mapping at tasks and subtask levels that reduces confusion during large enterprise project execution.

The approach described in this paper incorporates concepts elucidated by Ambler [2] that stress stakeholder involvement (and achieving better understand by stakeholder of the agile process, feedback, working (tested) functionality, change acceptance (by the developers), test driven development, flexible development teams, and adaptable incremental development. It is also important to note that many typical agile practices are not included in this particular implementation of the Bridge – light weight tools, fully self managing teams, dumping documents and just good enough documents [neither is acceptable in this organizational environment with intensive QA]. Thus, the agility of Bridge is that it approaches but does not exactly model the four values defined by the Agile Alliance. [1]

2. Research Methodology

An Action research methodology was used to expand and tailor the basic Bridge framework to the specific enterprise development project described in this paper. This choice of methods was dictated by the organization's objective of developing a system in a highly compressed time frame, and meeting the functional requirements of the system users. The stakeholders (members of a large government organization) sought immediate development results and clear outcomes measured against the sequential and traditional waterfall methodology milestones, as

well as rapid delivery of a robust and complex project. The project was complex in that it integrated a number of enterprise tools, developed a large amount of custom software, included approximately 125 full time project members (including contractors), and represented the largest single development project for the organization.

The user requirements addressed all aspects of the large project, and necessitated that significant changes occur at one time to develop the system using an Agile approach while still formally meeting all traditional milestones. Therefore, Action research seemed an appropriate methodology for this effort. This is consistent with Baskerville's [3] observation that Action researchers believe that complex social systems cannot be reduced, they must be considered as whole entities. Consequently, the best methodology for studying processes in these systems is to introduce changes and observe the results of these changes.

Susman and Evered [13] define Action research as consisting of five phases, all of which were included in the tailoring and implementation of the Bridge methodology. These consist of Diagnosing, Action Planning, Action Taking, Evaluating and Specifying Learning. These processes are iterated in a spiral type methodology. [13]

Baskerville [3] suggests that the ideal domain for Action research is one where the researcher is actively involved, where the knowledge obtained can be immediately applied, and where theory and practice are linked. In this Bridge project, those requirements are met since the work resulted in a cyclical process described by Susman & Evered [13], where new theories and practices were injected into the Bridge process during each iteration of the project.

The introduction of the Bridge methodology to the project and the participants occurred over a six week period. In the first planning phase (about two weeks) of the project's conversion to the Bridge methodology, the technical leader, and the team leaders held a series of meetings to determine project status, define objectives, discuss requirements, and share the differences between the expected project results and current performance. Previous work and actual results were examined, and perceived barriers to successful software development were identified.

The overall development status was recognized and agreed to by all. Stakeholders, managers, support staff, analysts, and developers concurred that many project milestones were not being accomplished. There was wide spread but unvoiced acceptance that the development process was unlikely to be successful within the established project deliverable dates. These perceived problems and responsibility for the project difficulties were broadly describes as being attributable

to the size and maturity of the organization (large, formalized, and not accustomed to rapid development), differing perception of executive guidance, lack of clarity of requirements, confusion over milestones, and changing requirements. All agreed (and reports showed) that major formal project milestones regarding project design work, documentation, and deliverables were either missed or significantly behind schedule.

The second phase of the Agile project initiation (two weeks) plan was prepared and implemented by the technical project manager to complete the documentation requirements and reviews required to achieve the missed formal documentation and review milestones. Concurrently, work began to subdivide the development effort into "chunks" that logically defined the project that would include all major products, specific functionality, and the framework for developing the project code. Finally, an assessment was made of the development teams' skills, and a search was begun to identify additional key technical staff (5-8 people) with the necessary expertise with the technologies to be used on the project.

The final stage of the initiation executed the formal reviews, delivered documentation, reorganized the teams and began Agile development following the Bridge process and methodology described below. (This work was extended by approximately one week due to "successes" and the holidays that occurred during this time period.)

At the end of each two week period, the activities were evaluated, and data collected was analyzed to determine if the processes put in place earlier were successful, and whether additional changes were required to the structure or the development effort. The processes used, seemed to realistically follow the iterative Diagnosis, Action Planning, Action Implementation, Evaluation and Learning steps needed to enable the group to focus on the longer term strategic goal of developing the system (using the Bridge methodology and Agile components previously described).

The Agile development process formally began approximately 7 weeks after its introduction to the project. The development and evolution of the Bridge methodology are discussed in detail below.

3. Bridge: the Development Environment

The overall Bridge development environment has three critical characteristics. First, it is Agile and deals well with changing requirements because of its focus on low level and iterative development and testing. Secondly, it meets the traditional reporting as well as

the schedule and control demands of management because the iterative development efforts can be mapped to traditional milestones, and because they generate performance metrics that report progress against defined objectives.

This paper argues that Bridge is Agile in that it implements agile concepts. Although these are not yet widely researched and published as measurement scales documented in the software methodology literature, they have been recognized and researched in a related manufacturing field. [29] Zhang and Sharifti assessed the underlying dimensions of agile capabilities with factor analysis and canonical discriminant analysis. Their taxonomy provides analytical evidence that there are three types of agility (in the manufacturing industries studied). The agilities stress differently the underlying concepts that make up agility (as a complex concept) and are: (1) the ability to satisfy and be close to customers, (2) the capability to thrive in changes that may be anticipated, and (3) the ability to cope with unanticipated changes. The value of the work in the manufacturing industry is to enhance competitive strategies so companies can tailor their approaches to the environments they face.

This paper further postulates that there may be multiple ways to be more (or less) Agile in software development, that the underlying concepts can be used strategically to deliver results (working systems), and that developers must recognize the environmental and organizational drivers of specific agile tools and techniques that are more (or less) useful. The specific mapping and linkages bridging the Agile process and the traditional method are found in four important areas.

First, requirements are determined early in the process and are stabilized at a high level throughout the life of the project. [Note; they will be mapped to specific development work during the development build processes, but this does not need to happen early in the process.] Secondly, major milestone deliverables associated with the traditional methodologies are produced and maintained. This includes the requirements reviews, system design specification, test plans, test readiness review, code deliveries, etc... Thirdly, Formal measurable and observable metrics traditionally used to proactively identify variances from intended baselines are supported in addition to trend data which provides a means of comparing project activities against known historical patterns. Finally, Formal methods tend to centralize risk management and hold upper level management responsible for preventing or mediating the identified risks while Agile methods leverage their rapid response times to reactively manage risks discovered opportunistically. Bridge methods drive all levels of

the program into active participation in risk management early, often and continuously.

Table 1. Build and traditional mapping

Strong Mappings	Unique Views	Primary User View
Requirements		Manager
Major Milestones		Manager
Deliverables		Manager
	Build Iterations	Developer
Performance metrics - tests		Manager & Developer
Build Performance		Developer

3.1 Requirements

The first stage is requirements gathering and analysis. Traditional methods seek to capture and document a complete understanding of the problem space as possible. The closer to 100% complete the less likely a discovered requirement will later derail the work in progress. Contrary to popular belief, Agile methods work well in this proactive requirement analysis phase. Light weight tools such as Use Cases, UML models, and boxes-and-lines architecture as well as highly cohesive customer involvement with the entire project team, not just the business analyst, provide the opportunity to create sets of self-validating models where any discovery made by one is quickly translated into the others. The three significant variances from the traditional and formal methods are: 1) Agile's early aggressive publication of products (and test results), and 2) Agile's willingness to be wrong and correct the defects identified via review in the next iteration. The third major difference is Agile's intention and ability to continue the requirements gathering and analysis process beyond the traditional requirements review milestones (throughout development and test) right up to final user acceptance. The hybrid Bridge process leverages the Agile differences when compared to the traditional methodology by: 1) reducing planned requirements identification and specification time; 2) initiating the iterative development process based on a Bridge daily cycle earlier in the project's life; and 3) shortening Quality Assurance and Acceptance testing phases more than the traditional Formal methods alone would have allowed because significant testing occurs during each daily cycle.

Bridging at the very beginning of the Software Development Lifecycle (SDLC) creates an environment of trust among the entire team. Stakeholders and oversight teams are directly involved

in the requirements discovery and validating process leading up to the definition of program scope. Program scope creep may therefore be monitored and formally approved (and time / budget adjusted appropriately) or disapproved. Instead of waiting for the program to surreptitiously report a creep or newly discovered need, the executives have the project SME's approach them to discuss Use Cases created from interviews conducted across the user population. Executives can thereby directly effect the course of activities well before the team spends an extensive amount of time decomposing, planning, estimating, and scheduling work for which there is no approved time and budget.

Similarly the team at the very lowest levels acquires knowledge in equal parts of the strategic direction set by the executives and the activities of the business users. Integrated teams including all of the roles; Program Leadership (PM) , Subject-Matter-Experts (SME), Test, Analyst, IV&V, Quality Assurance (QA), Development (Dev), Tech Writers, Security (Sec) and Project Leads (PjM), having equal access to all of the information early in the process and provides each community a chance to bring their perspective to the table.

Consider a Use Case describing a simple activity such as "Monitor an Account" for a bank. The initial Use Case describes a single actor, a Cashier, requesting a view of a customer's account. The PM might want information about min, max, and mean balances on a monthly basis; the Cashiers just want the current state and last transaction information; and the Security team levies a demand of complete auditability. Providing a lightweight Use Case early allows each of these perspectives the opportunity to judge the transaction's data completeness, identify missing user objectives and elements, or suggest a completely new (and highly valid) user requirement. The Testers and Developers can immediately add their perspectives thereby feeding testability and complexity back to the team. Taken together, with all of the users advancing their perspectives by reviewing, revising, and assessing, each use case increases in both detail and accuracy. In addition, this process identifies and resolves potential issues very early in the lifecycle resulting ultimately in reduced cost and time to system delivery and implementation.

Having involved everyone in identifying most, but not all, of the detailed requirements reduces overall risk. The Bridge methodology trades the promise of completeness usually associated with traditional and formal requirements gathering for more broadly reviewed requirements, and additional time between requirements acceptance and user acceptance. When contrasted with traditional approaches the value is significant. More time adds both slack and

management reserve to the development schedule. Traditional approaches can produce recurring reports and manage with little variance being shown against the original plan, thus serving program oversight and front office alike. The lack of detailed requirements will be simply tracked as a "risk" and mitigated by something akin to ongoing analyst activities. Agile approaches will leverage the time a bit differently. The integrated teams will break the program into smaller units of work. This might be phases comprised of releases and releases defined by iterations. Each iteration is very short, but still remains a complete lifecycle. This continues to keep the team focused on the delivery of product. However, the Agile approach may struggle meeting the needs of the executives without encumbering the team.

Bridging parts from both Formal and Agile practices creates an environment of complete transparency serving the oversight and executive offices, the front office and program management, and the product teams equally. Three things are required to be a successful Bridge across these perspectives:

- Reduced communication time between any two participants,
- a level of rigor that provides predictability and measures of progress, and
- an equal focus on "on-time" delivery and quality.

All three can be achieved in a project ruled by entropy. Projects with entropy will have highly flexible if not chaotic activities within rigorously defined and stable time and objective (functional performance) boundaries. This environment is created through the Daily Cycle.

3.2 Development Methodology: the Daily Cycle

The Daily Cycle begins as soon as the requirements, Use Cases, models, and other analytical artifacts are made available to the team. The problem, as it is presently described and understood, is broken into a loose set of parts aligning with the known Use Cases. The parts are bounded by the architecture and sequenced based on technical requirements, business requirements, and priorities fed back from the user community. The broad team is presented a subset of parts to do. They are given a chance to form self-managing teams around each part comprised of representatives from each role. If the right mix of roles is not available for a given part of the solution then either a resource filling that roles perspective is tasked across solution parts or that part is knowingly delayed (introducing risk) until the appropriate resources become available. The newly formed teams are

provided all of the known information including architecture, design, and requirements documents and the cycle begins.

Figure 1 below depicts the complete cycle.

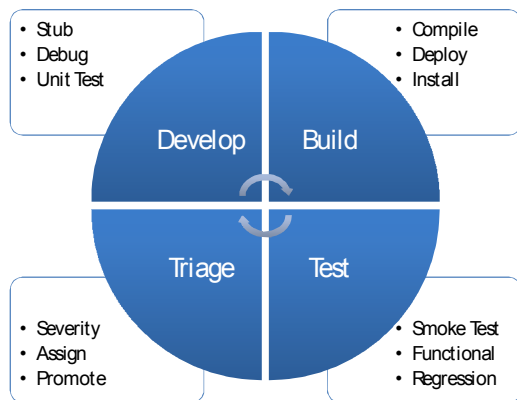


Figure 1. The Daily Cycle

3.3 Rosetta Stone Views

The net result is visually depicted as a cross-process Rosetta stone. (see figure 2) Each community sees the project in a language they are accustomed to side by side with the languages of the other community. The development team members see the detailed build results that help them overcome integration issues, and test results as defects assigned to them by the triage process. The project management team obtains the critical metrics needed to measure the development team's productivity, product quality, and performance against the predicted iteration and project development baseline. Oversight and program management can readily monitor progress toward milestones and delivery against budget. Ensuring total transparency by presenting the work status, metrics, and work products associated with every community side-by-side greatly improves understanding among the communities.

3.4 Development Processes

Development is comprised of Coding, Debugging, Unit Test, and Source Control processes. These are highly coupled non-sequential activities illustrated in figure 2 below.

Beginning with Development; the team, after reviewing the provided documents, works to refine the requirements created during project initiation and approved during a Critical Design Review process.

Traditionally lengthy searches to clarify requirements are dramatically reduced within and across empowered development teams. This occurs because of direct access to all roles and particularly those with Subject Matter Expertise.

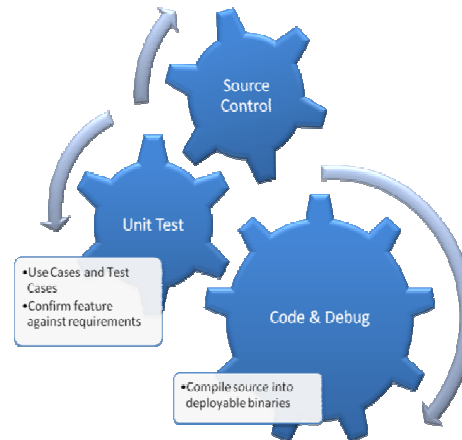


Figure 2. Development

Initial development activities are focused on “stubbing out” the working product. This exercise drives the teams to work with, and more importantly for each other. The teams need to actively negotiate across team boundaries, each deal creating an interface contract between the consumer and provider. This is entropy at a much lower level. After the teams come to agreement they must each trust the other to maintain a stable set of interfaces which are implemented and complied into the first build. An important secondary effect of this activity is the development of testable goals. The Test role embedded in each team incorporates the original requirements, additional information discovered during investigation and the negotiated needs of the teams they integrate with into a single set of Boolean testable requirements. The testable requirements are used by the developers to create unit tests for their piece of the solution and are integrated into the overall solution test suite.

Subsequent code development evolves from logic-less returns, to hard-coded dummy data returns, to its more substantive fully feature state, supporting production code. Incrementally fleshing out the details of each business feature as a team improves cohesion across roles. Feature teams, which on larger programs are likely to engage multiple development teams with multiple cross cutting activities, have clear and deliverable goals. This clarity and ownership tends to enhance a sense of pride in delivery and personal responsibility for product quality. As each feature nears completion, the team producing the feature must Unit test their part or parts. This is the first of several

opportunities to ensure completeness and quality but its value must be kept in context. Developer Unit Tests only verify that the developer has built what the developer believed he/she should build. Only independent tests by the test team will ensure the overall quality and completeness of the solution.

Measures

- Progress is measured by number of use cases and requirements remaining
- Quality is measured by unit tests passed and static code analysis
- Efficiency is measured by mean time to close defects

Measure	Before	After
use cases/requirements remaining	—	58/58*
unit tests passed	—	310/200**
mean time to close defects	—	2.3 days

* There were no use cases, user stories or narratives in the formal effort. The Bridge produced 68 Use cases of which 58 were in scope during this phase. All 58 were delivered.

** There were no unit test or test plans produced during the formal effort. The Bridge produced 310 unit tests with steps matching the details identified as necessary to pass the requirements they mapped to.

3.5 Build

Critical to the Agile development approach is a very stable Build/Deploy/Install process (collectively referred to as the Build) and process acceptance by the development team of Build “software control” that is essentially external to the developers. The process is initiated when any of the development teams believes its source code is ready to be added to the Build. It then checks the source for the component, the unit test, and necessary readme updates into the source control system, and informs the build master who will ensure it is included in the next release.

This is the first of several push/pull hand-offs. In large programs the developers do not know, and generally do not need to know, all of the contributing teams and their status relative to the next release, and by rule, the developers should never push their code directly into the Build. The Build process is, or more specifically the role of Build Master is, as program activities go, lonely, difficult, critical, and the easiest to point at when “bad things” happen. Maintaining version control, synchronizing dependant parts, and chasing down teams whose contribution has caused the Build to break are but a few never ending tasks for the Build Master.

This role is intentionally isolated from both Development and Test; the Build process creates a clean-room for the transformation of development outputs into deployable pieces. The build process must independently compile each and every component of the solution to eliminate any external dependencies that might have been added, intentionally or unintentionally, by the developer on their local machine. Next, the newly built pieces are moved from the output directories in the Build environment to their prospective target devices in a Development LAN. Once deployed, the Build Master installs the various pieces on their target device or devices. The more a solution leverages Commercial-off-the-shelf (COTS) software to fill out portions of their solution the more complex the Build process becomes. Each vendor’s server, tool, or technology has its own unique deployment and installation requirements, and the Build process is only successful when all parts are properly running on the target devices. [It should be noted that few managers have a complete appreciation for the complexity and difficulty of software enterprise builds until they are able to personally observe hours and days of laboratory build delays when developers have reported that “they are finished and their code is functioning correctly.”]

Measures

- Progress is measured for the Build as either Pass or Fail
- Quality is measured by the number of issues directly related to deployment and installation of the solution to each target environment
- Efficiency is measured by the total time beginning with a compile intended to be the next release and ends when that version of the solution is available to test.

Measure	Before	After
Build Pass/Fail	—	162/178*
Likelihood of Build Pass	—	65%
Deployment defects	—	11 **
Time Build to test	—	4 hours ***

* There were no Builds or compiled code created during the Formal effort. The Bridge Builds began very poorly becoming more consistently successful over time.

** The Bridge has averaged 11 defects per build/test cycle that are directly attributable to deployment and installation. This number has been falling as hardware configurations, and installation documentation has improved.

*** Time from Build to Test includes automated compile and test, deploying to a staging environment for developer smoke testing, creating media, logging and versioning, and deployment into the test environment.

3.6 Testing

Once the Build has installed all of the parts onto the Development LAN, the authors are notified to “smoke test” (verify that what they thought they completed was correctly included) their code. While not the most rigorous or conclusive test process, the Smoke Test is a critical activity to both the Daily Cycle and the overall product quality. This may be the first and only time the developers get a true integration test for this particular release of the solution. It will frequently expose interface and configuration issues otherwise invisible to an individual development team member.

Having passed the smoke tests the Build Master either deploys and installs the versioned components onto the Test LAN directly or creates installation media for the test team to perform an independent install. Much like the Build environment the Test environment is nearly sterile. The devices are frequently wiped and restored to a well known base image. Test rules are very strict. Only the Test team and system administrators have access. Any effort by a developer to “fix” a release which is blocking test is strictly prohibited in the test environment.

Initial releases to Test are intentionally incomplete. However as first class members of the development team the Testers have a detailed understanding of the solution before it reaches their test environment. They are responsible for creating the initial test plans defining the release scope for the developers and need to test each and every release as though it were complete. Defects identified are assigned one of five severities;

- Severity 1 defects cause complete solution failure stopping all tests. These require immediate attention by all. (Sev 1)
- Severity 2 defects block some test plans. These are serious defects and are the top priority of each assignee. (Sev 2)
- Severity 3 defects are not blocking and are characterized as causing the test plan to return an unpredicted result. These constitute the vast majority of defects found. (Sev 3)
- Severity 4 defects do not prevent the proper operation of the solution but should be addressed if possible. (Sev 4)
- NT defects are Not Testable in the current Build.

Releases tend to have a predicable flow associated with defects. Initially, many of the testable features will create NT defects with subsequent releases shifting to few NTs, a few Sev 2’s, and mostly Sev 3

defects. As the releases stabilize there will be no Sev 1, 2, or NT defects and all of the Sev 3 defects will be either resolved or a work-around identified.

It is critical that the test and development teams work closely together. During planning sessions the test team provides the perspective necessary to frame the work into Boolean testable components. This is translated into the tests to be passed as developer unit tests prior to submission to integration testing. This reduces developer injected scope creep, provides an open forum for SMEs to provide additional detailed information as part of a just-in-time design activity, and the team leads can identify and escalate integration issues quickly and early.

The working relationship can be assessed by first monitoring the velocity of each team. Velocity is defined as the teams’ average time required to produce high quality, complete units of functionality. Clarity as a result of the tester/developer/SME relationship results in a significant increase in that teams velocity. Ongoing effectiveness can be monitored with day over day tracking of Active, Resolved, and Closed defects. A balanced team will maintain a consistent flow of new Active, to developer Resolved, then finally tester Closed defects.

Measures

- Progress is measured as a function of coverage. The percentage of all Use Cases and for each Use Case the percentage of test cases executed.
- Quality is measured by monitoring the number of defects returned for definition over time and test coverage over time
- Efficiency is the total time required by the test team to fully execute all tests

Measure	Before	After
Coverage Tests automatic & manual	—	105/205*
Defects requiring definition	—	4%
Time required to test	—	2.5 days

* There was no testing during the Formal effort. The Bridge effort has yet to fully automate all tests.

3.7 Triage

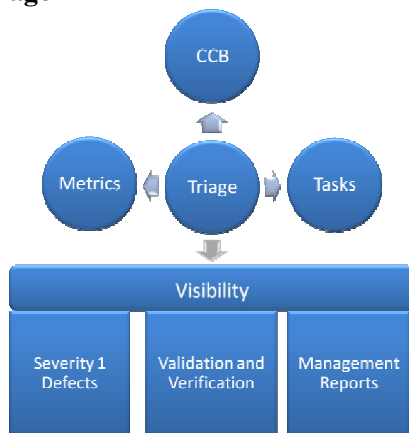


Figure 3: Triage

Defects identified by the test team in the current build are continuously added to the teams' defect tracking system as 'Active'. The Defect tracking system is a shared repository visible to all team members and is the sole input to the Triage process. The primary objective of the Triage process is to review and allocate defects to the appropriate team member. In most Agile environments this task would be done in near real time as developers identify a defect. They would either immediately resolve the defect or create a new unit test, and address the defect as a component of achieving a unit test. More traditional Formal approaches would have periodic reviews of Test results (multiple defects) presented to team management who would assign the issues to team members. Triage plays a broader role in a Bridged environment. In addition to providing a forum to assign tasks to developers, Triage creates an opportunity to;

- escalate defects to the Change Control Board,
- expose each defect to Quality Assurance, IV&V, Tech Writer, Usability, and Development
- ensure sev 1 defects are escalated immediately to key development resources
- generate management reports from near real time metrics

Measures

- Progress is measured by the percentage of Active defects assigned. Anything less than 100% requires immediate attention.
- Quality is measured by tracking the number of defects requiring re-assignment and clarification.
- Efficient Triage meetings should be one hour or less.

Measure	Before	After
% Active defects assigned	—	100%
Defects requiring re-assignment	—	20%
Average Triage Time	—	35 minutes

4. Value Proposition for this Bridge Process

This work used a Bridge process to manage and develop a large software system. The benefits are very strong, as shown in the table below.

Process Feature and Benefit Table

Requirements	Avoids differing views & incorporation of new and creeping requirements
Daily cycle	Promotes working prototypes, rapid corrections, iterative testing, and feedback
Rosetta Stone Views	Management, Developer and stakeholder process visibility
Development Processes	Provides immediate feedback loops for developers and Subject Matter Experts.
Build	Ensures 'clean' and repeatable product creation
Testing	Provides immediate feedback loops for testers and Subject Matter Experts.
Triage	Generates trusted metrics and increases current state-of-the-program visibility.

5. Conditions and Limitations

A number of different factors appear to be important to this Bridge approach.

First, tools must be available to provide: shared source code repository; build/deploy/install automation; collaborative environment for defect tracking; real-time test reporting; and collaborative communication within and across teams. The tools used in the project described in this research are shown in the table below.

Tool Usage

source code repository	Team Foundation Server
Build / Deploy / Install	Microsoft Installer packages (msi) and custom scripts

defect tracking	Serena Team Track
Real Time Test reporting	Serena Team Track
Collaborative communication	Email, Instant Messaging, SharePoint, Telephone

6. Acceptance

Applying Bridge methods was a significant change to all levels of the enterprise and while well received at the top and middle levels of the organization there were some notable exceptions within the lower levels. Front office management enjoyed the rapid feedback loop and quickly adopted Bridge practices. Middle and line managers were initially resistant as their traditional levels of influence were diminished and control moved down to the teams and individuals. Most but not all middle and line manager remained broad participants on the various teams and became key players enjoying the more intimate interaction across previously isolated organizations. Last, of the 53 people who were team members (Developers, Testers, QA, Usability, and SMEs) only 4 remain resistant to the Bridge approach.

7. Conclusions

The authors have presented a methodology that successfully integrated Agile with traditionally linear and sequential software development lifecycles using da Silva and da Cunha's process [10]. The methodology (Bridge) assembles a software and management environment serving multiple organizational purposes including project oversight, management control, and team planning and task scheduling. The new methodology and environment enables all project stakeholders to acquire a clear and constructive understanding of each other's activities. The methodology is tailored within the enterprise integration and software development project described in this paper, and implemented using meetings where key tasks required by project iterations are agreed to, and where it is determined that the project is ready to begin the next iteration of the system.

In the next phase of this work, the researchers will examine several hypotheses that have been advanced to "explain" the success of the Bridge methods. The objective of the work is to determine the extent, importance, and process through which the Bridge method introduces the "multiple supporting and self-validating views of the development process" appropriate to the perspective of the observer. In addition, the researchers seek to determine the process and extent to which the Bridge methodology aids the different users by creating a "mutual understanding of the different purposes of each perspective (traditional

and Agile)," and the extent to which the Bridge methodology supports the development of a "shared mapping" at task and subtask levels which reduces confusion during large enterprise project execution.

8. References

- [1] Alliance, Agile. "Manifesto for Agile Software". *Development*. <http://www.agilemanifesto.org/> (accessed September 2007).
- [2] Ambler, S. W., "The Agility Quotient", *Software Development*; 13 (1), (January 2005), pp. 52-53.
- [3] Baskerville, R. L., "Investigating Information Systems with Action Research", *Communications of the Association for Information Systems*, 2(19), October, 1999.
- [4] Baskerville, R. L., & Wood-Harper, A. T., "A critical perspective on action research as a method for information systems research", *Journal of Information Technology*, 1996, pp. 235-246.
- [5] Boehm, B., and Turner, R., *Balancing Agility and Discipline – A Guide for the Perplexed*. Boston: Addison Wesley, Person Education, 2004.
- [6] Boehm, B., "Requirements that Handle IKIWISI, COTS, and Rapid Change", *Computer, IEEE*, 2000, pp. 99-102.
- [7] Cockburn, A., *Surviving Object-Oriented Projects. The Agile Software Development Series*. Boston: Addison-Wesley, Person Education, 1998.
- [8] Eiseley, L., "The Hidden Teacher", in *the Unexpected Universe*. 1964-69: Harcourt, Brace Jovanovich.
- [9] Hull, E., Jackson, K., Dick, J., *Requirements Engineering – second edition*. Springer, 2004.
- [10] J. G. A. O. Basto da Silva, and P. R. da Cunha., "Reconciling the irreconcilable? A software development approach that combines Agile with Formal", *Proceedings of the 39th International Conference on System Sciences*, 2006.
- [11] Larman, C., *Agile & Iterative Development – A Manager's Guide. Agile software Development Series*. Addison Wesley, 2004.
- [12] Pressman, R. S., *Software Engineering*. New York: McGraw Hill, 2001.
- [13] Susman, G. I., & Evered, R. D., "An Assessment of the Scientific Merits of Action Research", *Administrative Science Quarterly*, 23(4), December, 1978: pp. 582-603.
- [14] Waxer, C., "Six Sigma-What is Six Sigma?". www.isixsigma.com/sixsigma/six_sigma.asp (accessed June 2007).